Autómatas, Teoría de Lenguajes y Compiladores

Proyecto Especial

Instituto Tecnológico de Buenos Aires

Grupo EZduino

Integrantes:

- Arias, Mateo (65613)
- Garcia Puente, Manuel (65505)
- González Núñez, Pedro (65613)
- Spivak Fontaiña, Federico (60538)

Fecha de Entrega: 03/04/2026

Índice

1. Equipo **3**
2. Repositorio **3**
3. Dominio **3**
4. Construcciones **3**
5. Casos de prueba **4**
6. Ejemplos **6**
6.1. Salida Digital (LED / Zumbador) 6
6.2. Entradas (Botón digital y Potenciómetro analógico) 6
6.3 Actuador Complejo (Servomotor) 7
6.4. Sensor Multipin (Ultrasónico HC-SR04) 7
6.5. Sensor Multivariable (DHT11 Clima) 8
6.6. Dispositivo Complejo de Salida (Pantalla LCD 16x2) 8

1. Equipo

Nombre	Apellido	Legajo	e-mail
Mateo	Arias	65613	matarias@itba.edu.ar
Manuel	Garcia Puente	65505	mgarciapuente@itba.edu.ar
Pedro	Gonzalez Nuñez	64467	pgonzaleznunez@itba.edu.ar
Federico	Spivak Fontaiña	60538	fespivak@itba.edu.ar

2. Repositorio

La solución será versionada en: 🔗 Github

3. Dominio

Desarrollar un lenguaje que permita facilitar la programación e implementación del código para poder usar los módulos más comunes de un Arduino UNO. El lenguaje desarrollado, denominado EZduino, busca abstraer la complejidad de la programación en C++ (gestión manual de pines, importación de librerías complejas, bloqueos por la función delay y la estructura obligatoria de setup y loop).

El lenguaje permite declarar de forma semántica los componentes físicos conectados a la placa (sensores ultrasónicos, pantallas LCD, módulos RFID, servomotores, etc.) y definir rutinas de comportamiento basadas en eventos o temporizadores que se ejecuten de manera continua y no bloqueante. La implementación de EZduino permitirá a hobbistas y desarrolladores concentrarse en la lógica de su circuito en lugar de lidiar con los detalles de bajo nivel del microcontrolador.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

(I). Se podrán declarar bloques de configuración de *hardware* para asignar componentes físicos a pines específicos de la placa.

(II). Se podrán crear bloques de *routine* que albergarán la lógica de ejecución continua.

(III). Las variables se declararán con la palabra clave var y podrán ser de tipo numérico (enteros / flotantes), booleanos, cadenas de texto o de tipo tiempo (ej. 200ms, 2s).

(IV). Se proveerán operadores relacionales clásicos (<, >, ==, !=, <=, >=).

(V). Se proveerán operadores lógicos (and, or, not).

(VI). Se proveerán estructuras de control condicionales de tipo if / else.

(VII). Se proveerán estructuras de control de tiempo no bloqueantes con la sintaxis *repeat every <tiempo>*, y *wait <tiempo>*, ambas diseñadas para no bloquear la ejecución del programa principal.

(VIII). Se proveerán métodos integrados específicos en inglés para interactuar con los componentes (ej. *.read_distance()*, *.print()*, *.move()*).

(IX). El bloque de *hardware* solo aceptará la declaración de los siguientes tipos de componentes: led, buzzer, button, potentiometer, servo, ultrasonic, dht11 y lcd.

(X). Se proveerán estructuras de iteración finita explícitas, permitiendo bucles de cantidad (*repeat <n> times*) y bucles de rango con variable iteradora (*for <var> from <inicio> to <fin>*).

5. Casos de prueba

Casos de prueba de aceptación:

(I). Un programa que declare un *led* en un pin digital en el bloque *hardware* y lo encienda estáticamente.

(II). Un programa que lea el estado de un *button* y, si está presionado, encienda un *buzzer* utilizando una estructura if / else.

(III). Un programa que lea un valor analógico mediante un *potentiometer* y lo asigne a una variable numérica.

(IV). Un programa que realice un barrido de un *servo* desde 0 hasta 180 grados utilizando una estructura de iteración de rango (for ... from ... to ...).

(V). Un programa que declare un sensor *ultrasonic* especificando sus pines *trig* y *echo*, y guarde la medición de *.read_distance()* en una variable.

(VI). Un programa que utilice un sensor dht11 para obtener tanto el valor de *.temperature()* como el de *.humidity().*

(VII). Un programa que inicialice una pantalla lcd (exigiendo la declaración de sus 6 pines correspondientes) y muestre un texto estático utilizando *.print().*

(VIII). Un programa que haga parpadear un *led* de forma asíncrona utilizando la estructura temporal *repeat every 500ms.*

(IX). Un programa que combine lógica condicional: active un *buzzer* solo si el *ultrasonic* detecta un objeto a menos de 10 cm.

(X). Un programa integral dentro del bloque *routine* que lea la temperatura del *dht11* y la imprima continuamente en la pantalla *lcd* refrescando los datos con *repeat every 2s.*

Casos de prueba de rechazo:

(I). Un programa malformado (errores léxicos o sintácticos generales por no respetar la gramática de EZduino).

(II). Un programa que intente utilizar un método de un componente en la *routine* sin haberlo declarado previamente en el bloque de *hardware*.

(III). Un programa que asigne dos componentes de hardware diferentes al mismo pin de la placa.

(IV). Un programa que intente realizar una operación matemática entre tipos de datos incompatibles (ej. sumar un texto a un tipo de dato de tiempo).

(V). Un programa que intente asignar un componente a un pin digital inexistente en la arquitectura del Arduino UNO (ej. pin 99).

6. Ejemplos

6.1. Salida Digital (LED / Zumbador)

```
// Palabras clave led o buzzer, un identificador y el pin.
hardware {
    led alert_light on 13;
    buzzer alarm on 8;
}

routine {
    alert_light.turn_on();
    repeat every 1000ms {
        alarm.toggle(); // prende y apaga
    }
}
```

6.2. Entradas (Botón digital y Potenciómetro analógico)

```
// Detectan estados (is_pressed()) o leen valores crudos de 0 a 1023.
hardware {
    button start on 2;
    potentiometer knob on A0;
}

routine {
    if start.is_pressed() == true {
        // La perilla devuelve un número entre 0 y 1023
        var speed = knob.read_value();
    }
}
```

6.3 Actuador Complejo (Servomotor)

```
hardware {
    servo barrier on 9;
}

routine {
    barrier.move(90); // Mueve el servo a 90 grados
    wait 2000ms; // Un delay controlado que no congele todo
    barrier.move(0);
}
```

6.4. Sensor Multipin (Ultrasónico HC-SR04)

```
// Como usa dos pines físicos, la declaración debe
// obligar a definir ambos.
// read_distance() retorna la distancia en centímetros (cm).
hardware {
    ultrasonic radar on (trig: 9, echo: 10);
}

routine {
    var current_distance = radar.read_distance();
    if current_distance < 20 {
        // Lógica de evasión
    }
}
```

6.5. Sensor Multivariable (DHT11 Clima)

```
// El objeto en el DSL tiene que proveer dos métodos distintos
// (temperatura y humedad) a partir de un solo pin de datos.
// temperature() retorna grados Celsius (°C).
// humidity() retorna porcentaje de humedad relativa (%).
hardware {
    dht11 climate_sensor on 4;
}

routine {
    var temp = climate_sensor.temperature();
    var hum  = climate_sensor.humidity();

    if temp > 30 {
        // Lógica para encender ventilador, por ejemplo
    }
}
```

6.6. Dispositivo Complejo de Salida (Pantalla LCD 16x2)

```
// Declaración de 6 pines obligatorios y métodos que requieren
// múltiples parámetros (fila, columna, texto).
hardware {
    // Declaración estricta de los pines rs, en, d4, d5, d6, d7
    lcd screen on (12, 11, 5, 4, 3, 2);
}

routine {
    screen.clear();
    screen.print(0, 0, "System Ready"); // Imprime en Fila 0, Columna 0
}
```